Model Capabilities & Limitations

What LLMs Can (and Can't) Do

Day 1 - Session 3

KavinSchool

Session 3: What We'll Cover

1. What LLMs do well: the "sweet spot"
2. Where LLMs fail: hallucinations and beyond
3. Knowledge cutoff and context window limits
4. Input preprocessing: cleaning, chunking, PII
5. Token counting and cost estimation
6. Model selection: capability vs cost tradeoff
7. **Demo:** Compare `gpt-4o-mini` vs `gpt-4o`

What LLMs Do Well

The Sweet Spot

KavinSchool

LLMs Excel At These Tasks

Task	Quality	Example
Summarization	Excellent	Condense a 10-page report into 3 bullets
Code Generation	Very Good	Generate boilerplate, CRUD, tests
Classification	Very Good	Spam detection, sentiment, intent routing
Rewriting/Editing	Excellent	Improve tone, fix grammar, simplify
Translation	Good	Not as good as DeepL for nuance, but fast
Q&A over given text	Excellent	Answer questions from a provided document
Format Conversion	Excellent	JSON → CSV, Markdown → HTML, SQL → ORM

Key pattern: LLMs are best when the answer is in the prompt or is a well-known pattern.

The "Easy Win" Prompts

Structured extraction is the most reliable use case:

```
messages = [
    {"role": "system", "content": "Extract the following fields from the "
     "text as JSON: name, email, company, role."},
    {"role": "user", "content": "Hi, I'm Sarah Chen from Acme Corp. "
     "I'm a Senior Engineer. Reach me at sarah@acme.com."},
]
```

The "Easy Win" Prompts - Response

Response:

```
{
  "name": "Sarah Chen",
  "email": "sarah@acme.com",
  "company": "Acme Corp",
  "role": "Senior Engineer"
}
```

> Structured output is deterministic and easily testable. Ideal for production.

Where LLMs Fail

Know the Limits Before You Ship

KavinSchool

The Five Failure Modes

1. Hallucinations

The model confidently states **false information** as fact.

```
Prompt: "Who invented the Python language and when?"
Output: "Python was invented by Guido van Rossum in 1991."  ✅

Prompt: "What paper did van Rossum publish in Nature in 2019?"
Output: "Van Rossum published 'Dynamic Type Systems at Scale'
         in Nature, Volume 573..."  ❌ (completely fabricated)
```

> **Rule:** Never trust LLM output for factual claims without verification.

The Five Failure Modes (continued)

2. Math & Logical Reasoning

```
Prompt: "What is 17 × 23?"
Output: "391" ✅ (sometimes right)

Prompt: "If I have 3 boxes with 7 items each, remove 4 from
         the first box, add 2 to the third, how many total?"
Output: [varies, often wrong on multi-step word problems]
```

3. Recent Events

```
Prompt: "Who won the 2025 Super Bowl?"
Output: [May give wrong answer or refuse, depends on cutoff]
```

> LLMs have a **knowledge cutoff date**. They don't know what happened after training.

The Five Failure Modes (continued)

4. Deterministic Logic

```
Prompt: "Is 'racecar' a palindrome?"
Output: "Yes" ✅

Prompt: "Is 'abcddcba' a palindrome?"
Output: "No" ❌ (it actually IS a palindrome. LLMs count characters poorly)
```

5. Long-Context Distraction

Even with 128K+ context windows, models can **lose information** in the middle of long inputs.

> **"Lost in the middle" problem:** Models attend best to the beginning and end of the context, less to the middle.

Hallucination Mitigation Strategies

Strategy	How	Effort
Provide source text	Include the facts in the prompt	Low
Ask for citations	"Quote the relevant passage"	Low
Temperature = 0	Reduce randomness	Low
RAG (Day 3)	Retrieve real documents, inject into prompt	Medium
Fact-checking chain	Second LLM call to verify claims	Medium
Fine-tuning	Train on your domain data	High

The #1 rule: If the answer matters, always provide the source material in the prompt.

Input Preparation

Clean Data In → Quality Output Out

Why Input Preparation Matters

> Garbage in, garbage out. This applies to LLMs more than any other technology.

Common input problems:

- **Too long:** exceeds context window or buries the question
- **Too noisy:** HTML tags, boilerplate, irrelevant text
- **Contains PII:** names, emails, SSNs sent to external API
- **Wrong format:** unstructured when structured would help

Input Preprocessing Checklist

```
def prepare_input(raw_text):
    # 1. Clean: remove noise
    text = strip_html_tags(raw_text)
    text = remove_boilerplate(text)

    # 2. Redact PII: protect privacy
    text = redact_emails(text)
    text = redact_phone_numbers(text)
    text = redact_names(text)  # if needed

    # 3. Chunk: split into manageable pieces
    chunks = chunk_text(text, max_tokens=2000)

    # 4. Select: pick only relevant chunks
    relevant = find_relevant_chunks(chunks, query)

    return relevant
```

Chunking Strategies

When input exceeds the context window, split it:

Strategy	How	Best For
Fixed-size	Split every N tokens	Simple documents
Sentence-based	Split at sentence boundaries	Narratives, articles
Paragraph-based	Split at paragraph breaks	Structured docs
Semantic	Split by topic/meaning	Complex documents
Overlapping	Include N tokens overlap between chunks	Preserving context

Chunking Strategies

```
# Simple fixed-size chunking
def chunk_text(text, max_tokens=2000, overlap=200):
    enc = tiktoken.encoding_for_model("gpt-4o")
    tokens = enc.encode(text)
    chunks = []
    for i in range(0, len(tokens), max_tokens - overlap):
        chunk_tokens = tokens[i:i + max_tokens]
        chunks.append(enc.decode(chunk_tokens))
    return chunks
```

Token Counting & Cost Estimation

Know What You'll Spend Before You Spend It

KavinSchool

Token Counting with `tiktoken`

```python
import tiktoken

def count_tokens(text, model="gpt-4o"):
    enc = tiktoken.encoding_for_model(model)
    return len(enc.encode(text))

# Count tokens for a full conversation
def count_conversation_tokens(messages, model="gpt-4o"):
    total = 0
    for msg in messages:
        total += count_tokens(msg["content"], model)
        total += 4  # overhead per message (role, delimiters)
    total += 2  # conversation framing
    return total
```

Token Counting with `tiktoken`

Example

```
messages = [
    {"role": "system", "content": "You are a Python expert."},
    {"role": "user", "content": "Explain decorators in detail."},
]
print(f"Input tokens: {count_conversation_tokens(messages)}")
# Input tokens: ~18
```

Cost Estimation

```
# Pricing per 1M tokens (as of 2025)
PRICING = {
    "gpt-4o":       {"input": 2.50, "output": 10.00},
    "gpt-4o-mini":  {"input": 0.15, "output": 0.60},
    "gemini-flash":  {"input": 0.10, "output": 0.40},
    "claude-sonnet": {"input": 3.00, "output": 15.00},
}

def estimate_cost(input_tokens, output_tokens, model="gpt-4o"):
    p = PRICING[model]
    cost = (input_tokens * p["input"] + output_tokens * p["output"]) / 1_000_000
    return f"${cost:.4f}"

# Compare: same prompt, different models
print(estimate_cost(500, 1000, "gpt-4o"))       # $0.0113
print(estimate_cost(500, 1000, "gpt-4o-mini"))   # $0.0007
print(estimate_cost(500, 1000, "gemini-flash"))   # $0.0005
```

> **16x cost difference** between GPT-4o and GPT-4o-mini for the same prompt!

Model Selection Decision Tree

```
Is the task simple (classification, extraction, formatting)?
├── YES → Use mini/flash model (cheap, fast)
│         GPT-4o-mini, Gemini Flash, Claude Haiku
│
└── NO → Does it require deep reasoning?
    ├── YES → Use capable model
    │         GPT-4o, Claude Sonnet, Gemini Pro
    │
    └── NO → Is data privacy critical?
        ├── YES → Use open-source, self-hosted
        │         Llama 3.1, Mistral
        │
        └── NO → Start with mini, upgrade if quality is too low
```

Demo Time

Same Prompt, Different Models

KavinSchool

Demo: Model Comparison

```
from shared import get_completion_full, get_model

prompt = "Explain the CAP theorem to a junior developer. Include a real-world example."

for tier in ["mini", "default"]:
    model = get_model(tier)
    response = get_completion_full(
        messages=[{"role": "user", "content": prompt}],
        tier=tier,
    )
    content = response.choices[0].message.content
    usage = response.usage
    print(f"\n{'='*60}")
    print(f"Model: {model} (tier={tier})")
    print(f"Tokens: {usage.prompt_tokens} in {usage.completion_tokens} out")
    print(f"{'='*60}")
    print(content[:500])
```

Watch for: depth of explanation, quality of examples, token usage.

Session 3: Key Takeaways

1. **LLMs excel at pattern-based tasks.** Summarization, extraction, classification
2. **LLMs fail at facts, math, and recent events.** Always verify
3. **Hallucinations are the #1 risk.** Provide source material in prompts
4. **Clean your input.** Remove noise, redact PII, chunk long text
5. **Count tokens before sending.** Know your costs
6. **Start cheap, upgrade if needed.** Mini models are 16x cheaper

> Next: Session 4, Hands-on Labs with User Story Generator & Bug Triager

Questions?

Session 3 Complete. Break Time!

KavinSchool